One text, two readers: compiling mathematical articles to Lean, sentence by sentence

Marcus Rockel

November 5, 2026

Department of Quantitative Finance,
Institute for Economics, University of Freiburg,
Rempartstr. 16, 79098 Freiburg, Germany,
marcus.rockel@finance.uni-freiburg.de

Abstract

When OpenAI announced in September 2026 that its agents had resolved the breakdown alternatives of the Navier–Stokes Millennium Prize Problem and released a Lean formalization, two questions dominated the reception: does the formal statement say what the problem says, and is the formal proof the proof in the paper? A passing Lean build answers neither, because the paper and its formalization are two separate texts. We present Proofread, which removes the second text. It compiles an ordinary LaTeX article, sentence by sentence and with fixed readings, to Lean: every definition becomes a Lean definition, every sentence of a statement a proposition, and every sentence of a proof a claim derived from the earlier sentences and the results it cites. The Lean kernel checks the compilation of the file itself, the identity of each readable statement with its compiled meaning, every justification, and a certificate whose citation audit forbids a proof step to use anything its sentence does not cite. The formal statement is therefore a function of the text, and the formal proof has the skeleton of the written one. Every term carries a definedness condition, so that no claim holds by a conventional value such as $1/0 = 0$. Sentences the grammar does not read remain prose, assert nothing, and are reported. A LaTeX package runs the check whenever the article compiles and marks every result in the margin with its status, and a web reader shows the formal reading of every sentence. We describe the design and its trust boundary, show how it addresses both questions on a worked example, report on two published research articles certified in full, and measure the coverage on an article not written for the tool.

1 Introduction

On 8 September 2026, OpenAI announced a proof that the three-dimensional incompressible Navier–Stokes equations with a smooth external force can break down in finite time, both on $\mathbb{R}^3$ and on the torus, and claimed to have settled alternatives (C) and (D) of the official problem description [26, 9]. The proof was produced by AI agents in a run of 88 hours [6], and it came with a Lean 4 formalization [27]. The formalization was the main reason for confidence: experts reported that the Lean code compiled, while the written paper was hard to follow, with one expert describing it as "not written for humans" [25].

Two questions followed, and they are the subject of this article.

(A) *Is the statement correct?* The Lean theorem is a proposition about Lean definitions of divergence, smooth initial data, forcing and solutions. Whether it expresses the Millennium problem is not something Lean checks.

(B) *Is the human-readable proof the Lean proof?* A reader learns the argument from the paper. If the Lean proof takes another route, the build certifies the theorem but not the paper.

Both questions turned out to be pertinent. For (A), the formalization is checked by Comparator [20] against two challenge theorems, one for $\mathbb{R}^3$ and one for the torus, using only the axioms `propext`, `Quot.sound` and `Classical.choice` [27]. Comparator certifies that a solution proves the very statement of a trusted challenge file, so it moves question (A) to that file without answering it. The challenge theorems, `navier_stokes_breakdown_R3` and `navier_stokes_breakdown_periodic`, carry the names of the statements in Google DeepMind's Formal Conjectures repository [10], on which the formalization says it builds. There, the divergence of a velocity field is defined through the Fréchet derivative, which Lean sets to 0 where a function is not differentiable, and the documentation flags the resulting junk value [12]. Statements of this kind are reviewed by experts, but they remain a second text whose agreement with the problem is a human judgment. For (B), Bastounis, Circelli and Hansen compared the Lean code with the written proof and found that they do not correspond: in a key estimate, the Lean lemma assumes one more derivative than the paper, so that Lean proves a strictly weaker statement than the one the written argument uses [4, 16]. They do not claim that the written proof is wrong. Their point is that a compiled formalization offers no assurance about the natural-language argument, and they argue that faithful translation of free mathematical text cannot be automated in general.

The underlying problem is structural. In the prevailing workflow, a paper and its formalization are two texts, written separately, by different people or by different runs of a model, and connected only by intent. Gap (A) lies between the two statements and gap (B) between the two proofs, and the kernel sees neither. PROOFREAD [32] closes both gaps by not having a second text. It reads the LaTeX source of the article itself. A fixed grammar assigns each sentence one reading, a total Lean function computes these readings from the file, and the kernel checks this computation, every proof, and a certificate. The trusted interface between the paper and Lean is then no longer a formalization of the paper, but a documented grammar and a short vocabulary, the same for every article.

Contributions.

- A design in which the formal statement is a function of the text (Section 3): sentences have fixed, documented readings; the compilation of the pinned source file is checked by the kernel; and each reading is displayed next to its sentence.
- Sentence-level derivation with a citation audit (Section 4): every proof sentence is a claim whose Lean justification may use only the earlier sentences of the proof, the results it cites, mathlib and the article. The Lean proof therefore has the skeleton of the written one.
- Definedness conditions for every compiled term, so that no claim is true by a conventional value of a partial operation.
- Tooling for authors and readers: a LaTeX package that checks the article whenever it compiles and marks every result in the margin, a web reader with the reading, Lean statement and premises of every sentence, and machine-readable reports.
- Experience (Section 7): example articles, two research articles certified in full [33, 1], a measurement of coverage on an article not written for the tool, and a discussion of who writes the justifications and at what cost (Section 6).

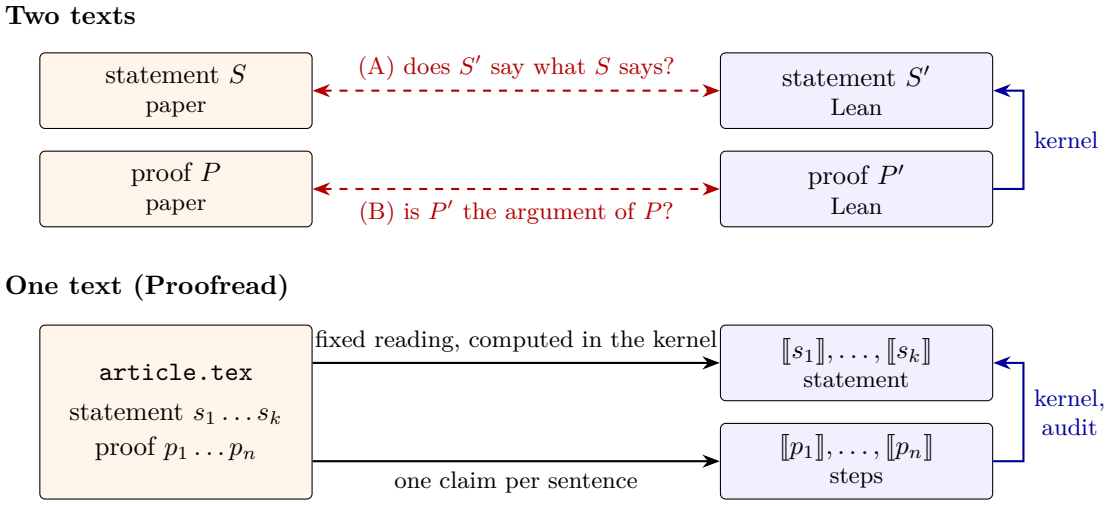


Figure 1: Top: in the usual workflow, the kernel checks that the formal proof P' proves the formal statement S'; the gaps (A) and (B) to the paper are not checked. Bottom: Proofread computes the readings $[\![\cdot]\!]$ of the sentences of the paper itself. Each step $[\![p_i]\!]$ is proved from its premises, the earlier steps and the cited results, and the statement from the steps; the kernel checks the compilation, the proofs and the audit.

2 Related work

Formal mathematics. Interactive theorem provers such as Lean 4 [8], with the library mathlib [37], now support the formalization of research mathematics, as in the Liquid Tensor Experiment [35]; see [2] for a survey of this "formal turn". In these projects, the formal development is written alongside the paper or after it, and the correspondence between the two is maintained by the people involved.

Autoformalization. Translating informal mathematics into formal statements and proofs by machine learning was proposed by Szegedy [36] and demonstrated with large language models by Wu et al. [41]; benchmarks such as ProofNet [3] followed, and a recent survey is [39]. Informal proofs can guide formal provers [15], and subgoal decomposition is central to recent provers [30]. Formal provers trained by reinforcement learning reach olympiad level [14], and agentic systems have completed large formalizations [24]. The faithfulness of an autoformalized statement is usually judged by humans, by back-translation, or by learned alignment scores such as FormalAlign [21], which estimate but do not certify agreement. Bastounis et al. [4] argue that faithful translation of free text is undecidable in general. Proofread sidesteps this limit by not translating free text: it reads a fixed fragment of mathematical English, on which the reading is a total, deterministic function, and it leaves everything else as reported prose.

Human-written statements and judges. Benchmarks such as Formal Conjectures [10] provide statements written and reviewed by experts, and Comparator [20] certifies that a solution proves exactly such a statement with permitted axioms. This settles agreement between two formal texts and leaves agreement between the formal statement and the mathematics to reviewers.

Linking papers and formalizations. Blueprints [22] link the statements of a LaTeX document to Lean declarations by name; a `\leanok` marks an item as formalized on the author's word, and the tooling checks that the named declarations exist. LeanArchitect

Approach	(A) formal statement vs. mathematics	(B) formal proof vs. written proof
Autoformalization with a model	model output, reviewed by humans or scored [21]	not related [4]
Reviewed statement and judge [10, 20]	human review of the statement; the judge compares formal with formal	not applicable (no written proof)
Blueprint [22, 42]	declarations named by the author; existence checked	dependency graph, asserted by the author
Readable formal languages [13, 40, 23]	the formal text is the statement	the formal text is the proof, in a formal syntax
Controlled natural language [38, 7]	by construction, in the controlled language	by construction; steps by automated provers
PROOFREAD	the fixed reading of the sentence, computed in the kernel from the pinned file and displayed	each sentence a claim; citation audit; prose reported

Table 1: How approaches relate the formal statement and proof to the mathematics and the written proof.

[42] generates blueprint content from annotated Lean code, which keeps the two in sync and surfaced inconsistencies in existing blueprints, but it does not verify that the informal statements match the formal ones. sTeX [18] adds semantic markup to LaTeX. In the other direction, Informath [29] generates fluent natural language from formal mathematics with the Grammatical Framework [28]. PROOFREAD differs from all of these in that the informal text is the source and the formal statements are computed from it, with no markup and no second text.

Readable formal languages and natural-language proof checking. Mizar [13] and Isabelle/Isar [40] were designed to make formal proofs readable, and Verbose Lean [23] gives Lean tactics a controlled natural-language syntax. SAD and its language ForTheL [38], and their successor Naproche [7], check texts in a controlled natural language embedded in LaTeX with automated provers; they are the closest predecessors of PROOFREAD. Controlled natural languages are surveyed in [19], and the linguistics of mathematical text is analyzed in [11]. PROOFREAD adds to this line four things: the target is Lean with mathlib and its kernel, with a kernel-checked compilation of the source file; each step may be justified by any Lean proof, subject to the citation audit; every term carries a definedness condition; and an ordinary article, the one that is submitted, may mix compiled sentences with prose, whose extent is reported. The idea of a single source for two audiences goes back to literate programming [17].

Table 1 summarizes how the approaches address the two gaps.

3 Design

3.1 From the source file to a certificate

An article is an ordinary LaTeX file. PROOFREAD proceeds in four stages, each a Lean command whose result the kernel checks.

Compile.

`#proofread_article` reads the file and evaluates `compileArticle`, an ordinary total Lean function, on its contents. The kernel checks the result by `Eq.refl` on literals, stage by stage (preamble, lexing and segmentation per block, sentences in chunks, assembly), without `native_decide`. The command takes a hash of the file as an

Construction	Example (from the documentation)
class	Let $\mathcal{C}$ denote the class of bivariate copulas, that is, distribution functions on $[0,1]^2$ with uniform marginals.
quantity	For a copula C, the population version of Blest's rank correlation is $\nu(C) \coloneqq 24 \int_0^1 \int_0^1 (1-u)\, C(u,v)\, \mathrm{d}u\, \mathrm{d}v - 2$.
map	Define $f : (0,\infty) \to \mathbb{R}$ by $f(x) \coloneqq x + \frac{1}{x}$.
notion	We call a copula C *positively quadrant dependent* (*PQD*) if $C(u,v) \geq uv$ for all $u, v \in [0,1]$.
claim	Every $C \in \mathcal{C}$ satisfies $\lvert\nu(C) - \rho(C)\rvert \leq \frac{1}{4}$.
exact region	It holds that $\{(\rho(C), \nu(C)) : C \in \mathcal{C}\} = \{(r,n) \in [-1,1]^2 : 2r - \Phi(r) \leq n \leq \Phi(r)\}$.
proof step	By Lemma 2.1, $f(x) - 2 = \frac{(x-1)^2}{x} \geq 0$.

Table 2: Some constructions of the grammar. Each has one reading, documented next to its parser.

argument, so an edited article is recompiled, and every report is pinned to the text it was computed from.

State.
`#proofread_semantics` turns every compiled definition into a Lean definition named after its symbol, and every claim K into a proposition `claimK`, its definedness condition `meaningfulK`, and the proof obligation `obligationK`, each tied to the compiled document by a kernel-checked `rfl` theorem.

Justify.
The obligations are proved in ordinary Lean, in a separate file, by people or by machines (Section 6).

Certify.
`#proofread_certify` checks that every justified claim holds, that its premises are verified and acyclic, that every definition is well defined, that claims derived in the text pass the citation audit, and that only the axioms `propext`, `Classical.choice` and `Quot.sound` are used. It exports `verifiedK : meaningfulK ∧ claimK` for every verified claim and writes the reports.

3.2 Sentences and their readings

The body of the article is split into sentences, each with its environment (running text, a result, or a proof). A sentence is compiled if it matches one of the grammar's constructions, and each construction has one documented reading. Table 2 shows a few. Every symbol must be defined before it is used, and a definition is its only meaning. Preamble macros are expanded first, so the author's notation is kept. A sentence that matches no construction is *prose*: it asserts nothing checked. Prose is welcome in the running text (motivation, remarks, sketches), and inside a statement it is reported, because that part of the statement is not checked.

3.3 Definedness

Partial operations are total in Lean and mathlib: $1/0 = 0$, and a derivative is 0 where it does not exist [5]. Such conventions are harmless in a library and dangerous in a statement, because a claim can then hold for a conventional reason. Every compiled term therefore has a syntactic definedness condition, and a claim means *defined and true*. The condition includes nonzero denominators, nonnegative bases of real powers, arguments in the domain

a definition states, integrability of integrands, measurability of measured sets, coverage of case distinctions, and the existence and uniqueness of the object a definite description names. Every definition must be proved well defined.

3.4 Claims, premises and the citation audit

The premises of a claim are the results its sentence cites and, for a sentence of a proof, the earlier sentences of the proof. Its obligation is

$$\texttt{obligationK} \ : \ P_{j_1} \to \cdots \to P_{j_n} \to \texttt{meaningfulK} \wedge \texttt{claimK},$$

where $P_{j_1}, \ldots, P_{j_n}$ are the premises. A claim is *derived in the text* if it is a proof sentence, or a statement whose proof compiled; its premises are then the sentences of that proof. For such claims the certificate audits the justification: every constant it uses must come from Lean, mathlib, PROOFREAD, the compiled article, or helper declarations that are themselves audited. A proof step therefore cannot be justified by a result its sentence does not cite. A claim without a compiled proof, such as a cited theorem, may be justified by any Lean proof, typically by a companion development; such claims are reported as *checked by a companion development*. A claim without a justification is *unverified*, and so is every claim that rests on it. A false claim is *refuted* by a proof of `meaningfulK` $\wedge\neg$`claimK`, and claims resting on it are not verified.

3.5 Trust boundary

The kernel checks the compilation of the file, the identity of the readable definitions and claims with the compiled semantics, every justification and the certificate. Outside the kernel's proof are the readings of the grammar's constructions (fixed and documented), the vocabulary and its model (for articles on copulas, four constants: distribution functions on the unit square, uniform marginals, probability measures on the square and their uniform marginals), the native evaluation that proposes the result of each stage (which the kernel then re-checks), and the presentation of the readings.

4 How the two gaps close

4.1 (A) The statement is the reading of the sentence

There is no formal statement besides the reading of the sentence. What remains to be trusted is the reading, and the reading is shown: the checker prints the reading of every claim as a formula, the web reader displays it next to the sentence, and the definitions it rests on are the article's own, compiled from the article's own sentences. The external definitions are those of the vocabulary, which is short and reviewed once for all articles that use it, in contrast to challenge files whose custom definitions must be reviewed for every problem. Definedness conditions exclude junk-value truths of the kind flagged in Section 1. And a false statement is caught when its proof fails or when it is refuted, as in Section 5.

4.2 (B) The Lean proof has the skeleton of the written proof

Let $p_1, \ldots, p_n$ be the sentences of a compiled proof of a statement s. Each $[\![p_i]\!]$ is a Lean claim whose justification may use only the $[\![p_j]\!]$ with $j < i$ and the results p_i cites, besides mathlib, and $[\![s]\!]$ is justified from the $[\![p_i]\!]$ and the citations alone. The formal proof is thus a sequence of local proofs, one per sentence, in the order of the text, and the audit forbids any of them to reach for an argument the text does not give. Within a single step, the Lean

proof may add detail: a step such as "Then $f(x) - 2 = x + \frac{1}{x} - 2 = \frac{x^2-2x+1}{x} = \frac{(x-1)^2}{x}$" is justified by `field_simp` and `ring`. The correspondence between written and formal proof is therefore exact at the level of sentences, which is the level at which a reader checks a proof. A proof that does not compile leaves its statement checked by a companion development, which answers (A) but not (B), and the reports say so.

5 A worked example

The tutorial of PROOFREAD follows a one-page note on the function $f(x) = x + \frac{1}{x}$ on $(0, \infty)$. Its central result reads:

```
\begin{theorem}\label{thm:bound}
For every $x\in(0,\infty)$, $f(x)\ge2$.
\end{theorem}

\begin{proof}
Let $x\in(0,\infty)$.
Then $\frac{(x-1)^2}{x}\ge0$.
By Lemma~\ref{lem:identity}, $f(x)-2=\frac{(x-1)^2}{x}\ge0$.
\end{proof}
```

The statement compiles to claim 2, $\forall x \in \mathbb{R},\ 0 < x \Rightarrow 2 \le f(x)$. The first proof sentence binds x; the second compiles to claim 3, $0 \le \frac{(x-1)^2}{x}$; and the third, which cites the lemma (claim 0), compiles to claim 4, the conjunction $f(x) - 2 = \frac{(x-1)^2}{x} \wedge 0 \le \frac{(x-1)^2}{x}$. The premises of claim 4 are claims 0 and 3, and those of claim 2 are claims 3, 4 and 0. The justifications are short:

```
theorem proof4 : obligation4 := by
  intro_premises
  exact ⟨by unfold meaningful4; definedness [], fun x hx => ⟨c0 x hx, c3 x hx⟩⟩

theorem proof2 : obligation2 := by
  intro_premises
  refine ⟨by unfold meaningful2; definedness [], fun x hx => ?_⟩
  obtain ⟨h, h'⟩ := c4 x hx
  linarith
```

Here `c0`, `c3` and `c4` are the premises, and the audit confirms that nothing else of the article is used. All seven claims of the note are derived in the text.

The tutorial's draft of the same note contains four mistakes of the kind that reach referees. A typo in a proof (g for f) is an undefined symbol. A definition on all of $\mathbb{R}$ fails its well-definedness condition at $x = 0$. A corollary stated loosely ("f attains its minimum on $(0, \infty)$ at 1") has no reading and is reported. And the theorem states $f(x) > 2$, which is false at $x = 1$: its proof does not go through, and a three-line refutation records that it is false as stated. The certificate then marks the theorem false and the corollary, whose proof cites it, as not verified, although that proof is correct (Fig. 2). The LaTeX package shows the same statuses in the margin of the typeset article (Fig. 3); without its `status` option the typeset article is unchanged.

6 Who writes the justifications

PROOFREAD computes statements, not proofs: every compiled claim becomes an obligation that someone must prove. It is natural to ask whether this makes verification more expensive than asking a person or a model to prove the theorems of a paper directly. We argue that it does not.

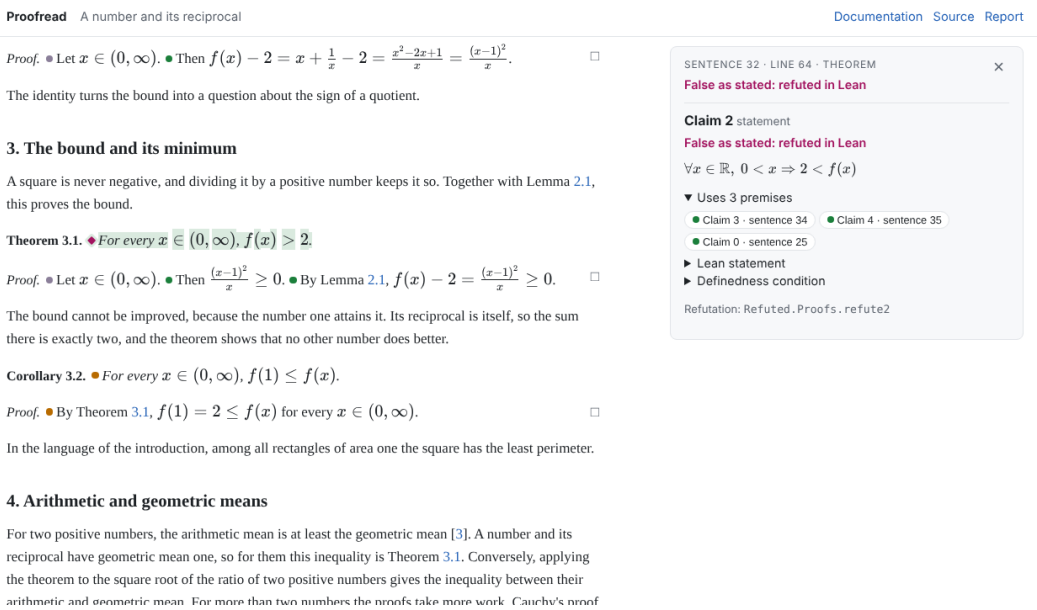


Figure 2: The web reader of the draft. Theorem 3.1 is selected: its reading, the three premises it rests on, and the refutation.

1. The statements cost nothing. No model formalizes them, and nobody needs to check that a model did so faithfully, which was the expensive and unreliable part in Section 1.

2. For a result whose proof compiled, the paper has already decomposed the argument. Each obligation is one step, from the previous steps and the cited results, and the obligation of the result itself is glue. This is the decomposition that guided and subgoal-based provers try to reconstruct [15, 30]; here the author supplies it.

3. For a result whose proof stays prose, the obligation is the statement itself, which is the baseline, and the result is reported as checked by a companion development.

The worst case is therefore the baseline, and the usual case is cheaper. This suggests a tiered procedure. First, a fixed portfolio of tactics (`linarith`, `nlinarith`, `positivity`, `norm_num`, `field_simp` followed by `ring`, and general automation) is tried on every obligation without any model; in the example of Section 5, every obligation closes in one to four lines of such tactics. Second, a model receives the remaining obligations one at a time, in parallel, with the sentence, its premises and the surrounding proof as context: small, local goals rather than theorems. Third, a step that resists both is a candidate for a refutation or a gap in the written proof, and the remedy is a further sentence in the paper rather than a longer Lean proof. Proofs are cached by the Lean statement they prove, so an edit to the article only re-proves what changed. None of this enters the trusted path: whoever writes a justification, the kernel and the audit check it.

7 Experience

Examples. The documentation contains four short articles in the usual style, each with an introduction and references, certified in continuous integration: the note of Section 5 (7 claims, all derived in the text), its draft with the false theorem, an article on the harmonic, geometric and arithmetic mean (16 claims, all verified), and an article showing that tail-monotone copulas are positively quadrant dependent (8 claims, all derived in the text).

Lemma 2.1. *For every* $x \in (0,\infty)$, $f(x) - 2 = \frac{(x-1)^2}{x}$. ✓ derived

Proof. Let $x \in (0,\infty)$. Then $f(x) - 2 = x + \frac{1}{x} - 2 = \frac{x^2-2x+1}{x} = \frac{(x-1)^2}{x}$. □

The identity turns the bound into a question about the sign of a quotient.

3 The bound and its minimum

A square is never negative, and dividing it by a positive number keeps it so. Together with Lemma 2.1, this proves the bound.

Theorem 3.1. *For every* $x \in (0,\infty)$, $f(x) > 2$. × false

Proof. Let $x \in (0,\infty)$. Then $\frac{(x-1)^2}{x} \geq 0$. By Lemma 2.1, $f(x) - 2 = \frac{(x-1)^2}{x} \geq 0$. □

The bound cannot be improved, because the number one attains it. Its reciprocal is itself, so the sum there is exactly two, and the theorem shows that no other number does better.

Corollary 3.2. *For every* $x \in (0,\infty)$, $f(1) \leq f(x)$. ○ compiled

Proof. By Theorem 3.1, $f(1) = 2 \leq f(x)$ for every $x \in (0,\infty)$. □

In the language of the introduction, among all rectangles of area one the square has the least perimeter.

Figure 3: The typeset draft with `\usepackage[status]{leanproof}`: Lemma 2.1 is derived, Theorem 3.1 is false, Corollary 3.2 is compiled but not verified.

Research articles. Two research articles on exact regions of rank correlations are compiled and certified in full [31]: an article on Blest's rank correlation [33], with 37 definitions and 48 claims, and an article on Spearman's rho and Gini's gamma [1], with 11 definitions and 8 claims. In the first, the proof of one corollary is checked sentence by sentence; the other results are proved through bridge theorems to an existing Lean development, so that (A) holds for all results and (B) for that corollary. Each certified article comes with a record of every edit the conventions required. For the first article, with 351 sentences, the kernel-checked compilation takes about 70 seconds with the lexing cache and peaks at 2.6 GB of memory; after an edit, only the changed sections are relexed.

An article not written for the tool. To measure coverage, we ran the checker on a third research article [34] as submitted, 576 sentences long. Only 2 sentences compiled at first. The cause was concentrated: five coefficients were defined inline with $=$ instead of $:=$, or in sentences that also carried citations and further definitions, and everything built on them failed in turn. After rewriting the seven defining passages in the documented forms, 9 of 587 sentences compiled, and the check reported 92 findings. Of these, 29 were prose in remarks and 15 repeated an undefined symbol already reported at its first use, so that reporting only genuine gaps in statements and each missing definition once leaves 48. The remaining findings point to vocabulary and constructions the grammar does not have yet: partial derivatives of copulas, shuffles of min, couplings and Markov kernels, statements of attainment ("attained by … and by no other copula"), and case distinctions in proofs. This is the current frontier of the tool, and it shows the intended mode of growth: each construction is added once, with its reading, for all articles.

8 Limitations

Proofread checks what it reads, and it reads a fragment of mathematical English that grows with use; an article not written with its conventions in mind compiles only partially, and its unread statements are reported rather than checked. The readings of the constructions are trusted: they are fixed and documented, but an author must read the

reading of a claim once to confirm that it says what was meant. The vocabulary and its model are trusted in the same way. The correspondence of proofs is exact at the granularity of sentences, and within a sentence the Lean proof may use any argument from mathlib. The kernel-checked compilation of long articles takes time and memory, which the incremental compilation reduces after edits. Finally, PROOFREAD certifies agreement between an article and Lean, not the significance or originality of the mathematics.

9 Conclusion

The reception of the Navier–Stokes formalization showed that a compiled Lean proof settles less than it seems as long as it is a second text next to the paper. PROOFREAD makes the paper the source: its statements are computed from its sentences, its proofs are checked sentence by sentence against what they cite, and the parts it does not read are reported. If AI systems are to write mathematics for people, as they increasingly do, a paper compiled in this way would answer both questions of Section 1 for every result it covers, and would say clearly which results it does not.

Availability. PROOFREAD, its LaTeX package, the web reader, the documentation and the example articles are available at https://github.com/Corrram/proofread, with a live demo at https://corrram.github.io/proofread/.

References

[1] Jonathan Ansari, Marcus Rockel, and Stefanie Steinmaßl. The exact region determined by Spearman's rho and Gini's gamma. *arXiv preprint arXiv:2609.19890*, 2026.

[2] Jeremy Avigad. Mathematics and the formal turn. *Bulletin of the American Mathematical Society*, 61(2):225–240, 2024.

[3] Zhangir Azerbayev, Bartosz Piotrowski, Hailey Schoelkopf, Edward W. Ayers, Dragomir Radev, and Jeremy Avigad. ProofNet: Autoformalizing and formally proving undergraduate-level mathematics. *arXiv preprint arXiv:2302.12433*, 2023.

[4] Alexander Bastounis, Fabian Circelli, and Anders C. Hansen. Navier–Stokes lost in translation: Why Lean verification of AI autoformalisation does not guarantee correct natural language proofs. *arXiv preprint arXiv:2610.08144*, 2026.

[5] Kevin Buzzard. Division by zero in type theory: a FAQ. Xena project blog, https://xenaproject.wordpress.com/2020/07/05/division-by-zero-in-type-theory-a-faq/, 2020.

[6] CNBC. OpenAI claims to have solved the 90-year-old Navier–Stokes math problem in 88 hours. https://www.cnbc.com/2026/09/09/openai-navier-stokes-math-problem-solved.html, 2026. 9 September 2026.

[7] Adrian De Lon, Peter Koepke, Anton Lorenzen, Adrian Marti, Marcel Schütz, and Makarius Wenzel. The Isabelle/Naproche natural language proof assistant. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 614–624. Springer, 2021.

[8] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021.

[9] Charles L. Fefferman. Existence and smoothness of the Navier–Stokes equation. In James Carlson, Arthur Jaffe, and Andrew Wiles, editors, *The Millennium Prize Problems*, pages 57–67. Clay Mathematics Institute and American Mathematical Society, 2006.

[10] Moritz Firsching, Paul Lezeau, Salvatore Mercuri, Miklós Z. Horváth, Yaël Dillies, Calle Sönne, Eric Wieser, Fred Zhang, Thomas Hubert, Blaise Agüera y Arcas, and Pushmeet Kohli. Formal Conjectures: An open and evolving benchmark for verified discovery in mathematics. *arXiv preprint arXiv:2605.13171*, 2026.

[11] Mohan Ganesalingam. *The Language of Mathematics: A Linguistic and Philosophical Investigation*, volume 7805 of *Lecture Notes in Computer Science*. Springer, 2013.

[12] Google DeepMind. Existence and smoothness of the Navier–Stokes equation (Formal Conjectures, file `millenium/navierstokes.lean`). https://google-deepmind.github.io/formal-conjectures/doc/FormalConjectures/Millenium/NavierStokes.html. Accessed 5 November 2026.

[13] Adam Grabowski, Artur Korniłowicz, and Adam Naumowicz. Mizar in a nutshell. *Journal of Formalized Reasoning*, 3(2):153–245, 2010.

[14] Thomas Hubert et al. Olympiad-level formal mathematical reasoning with reinforcement learning. *Nature*, 2025. doi: 10.1038/s41586-025-09833-y.

[15] Albert Q. Jiang, Sean Welleck, Jin Peng Zhou, Wenda Li, Jiacheng Liu, Mateja Jamnik, Timothée Lacroix, Yuhuai Wu, and Guillaume Lample. Draft, sketch, and prove: Guiding formal theorem provers with informal proofs. In *International Conference on Learning Representations (ICLR 2023)*, 2023.

[16] King's College London. OpenAI's Navier–Stokes proof called into question. https://www.kcl.ac.uk/news/openais-navier-stokes-proof-called-into-question, 2026. News release, 27 October 2026.

[17] Donald E. Knuth. Literate programming. *The Computer Journal*, 27(2):97–111, 1984.

[18] Michael Kohlhase. Using LaTeX as a semantic markup format. *Mathematics in Computer Science*, 2(2):279–304, 2008.

[19] Tobias Kuhn. A survey and classification of controlled natural languages. *Computational Linguistics*, 40(1):121–170, 2014.

[20] Lean FRO. Comparator: a trustworthy judge for Lean proofs. https://github.com/leanprover/comparator. Merged into Lean core as of release v4.35.0; accessed 5 November 2026.

[21] Jianqiao Lu, Yingjia Wan, Yinya Huang, Jing Xiong, Zhengying Liu, and Zhijiang Guo. FormalAlign: Automated alignment evaluation for autoformalization. In *International Conference on Learning Representations (ICLR 2025)*, 2025.

[22] Patrick Massot. leanblueprint: a plasTeX plugin for writing blueprints for Lean 4 projects. https://github.com/PatrickMassot/leanblueprint, 2026. Software, accessed 5 November 2026.

[23] Patrick Massot. Verbose Lean 4. https://github.com/PatrickMassot/verbose-lean4, 2026. Software, accessed 5 November 2026.

[24] Math Inc. Gauss. https://www.math.inc/gauss, 2025. An agent for autoformalization, used to complete a Lean formalization of the strong prime number theorem.

[25] NPR. Mathematicians learn little from AI completing unsolved problem. https://www.npr.org/2026/09/22/nx-s1-5968588/openai-navier-stokes-problem-mathematicians-learn-little, 2026. 22 September 2026.

[26] OpenAI. On the Navier–Stokes Millennium Prize Problem. https://openai.com/index/navier-stokes-solution/, 2026. Announcement of 8 September 2026, updated 10 September 2026.

[27] OpenAI. Finite time blowup for Navier–Stokes and Euler equations: Lean 4 formalizations. https://github.com/openai/NavierStokesAndEuler, 2026.

[28] Aarne Ranta. Translating between language and logic: What is easy and what is difficult. In *Automated Deduction – CADE-23*, volume 6803 of *Lecture Notes in Computer Science*, pages 5–25. Springer, 2011.

[29] Aarne Ranta. Symbolic informalization: Fluent, productive, multilingual. *arXiv preprint arXiv:2606.16893*, 2026.

[30] Z. Z. Ren et al. DeepSeek-Prover-V2: Advancing formal mathematical reasoning via reinforcement learning for subgoal decomposition. *arXiv preprint arXiv:2504.21801*, 2025.

[31] Marcus Rockel. lean-verifications: Lean 4 supplements to articles on copulas. https://github.com/Corrram/lean-verifications, 2026.

[32] Marcus Rockel. Proofread: mathematical articles in LaTeX, compiled and checked by Lean. https://github.com/Corrram/proofread, 2026.

[33] Marcus Rockel. Rank weighting and asymmetry in Blest's rank correlation: two exact regions. *arXiv preprint arXiv:2609.27634*, 2026.

[34] Marcus Rockel. Exact regions for the symmetrized Blest coefficient, 2026. arXiv preprint.

[35] Peter Scholze. Liquid Tensor Experiment. *Experimental Mathematics*, 31(2):349–354, 2022.

[36] Christian Szegedy. A promising path towards autoformalization and general artificial intelligence. In *Intelligent Computer Mathematics (CICM 2020)*, volume 12236 of *Lecture Notes in Computer Science*, pages 3–20. Springer, 2020.

[37] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pages 367–381, 2020. doi: 10.1145/3372885.3373824.

[38] Konstantin Verchinine, Alexander Lyaletski, and Andrei Paskevich. System for automated deduction (SAD): A tool for proof verification. In *Automated Deduction – CADE-21*, volume 4603 of *Lecture Notes in Computer Science*, pages 398–403. Springer, 2007.

[39] Ke Weng, Lun Du, Sirui Li, Wangyue Lu, Haozhe Sun, Hengyu Liu, and Tiancheng Zhang. Autoformalization in the era of large language models: A survey. *arXiv preprint arXiv:2505.23486*, 2025.

[40] Markus Wenzel. Isar – a generic interpretative approach to readable formal proof documents. In *Theorem Proving in Higher Order Logics (TPHOLs'99)*, volume 1690 of *Lecture Notes in Computer Science*, pages 167–183. Springer, 1999.

[41] Yuhuai Wu, Albert Q. Jiang, Wenda Li, Markus N. Rabe, Charles Staats, Mateja Jamnik, and Christian Szegedy. Autoformalization with large language models. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, 2022.

[42] Thomas Zhu, Pietro Monticone, Sean Welleck, and Jeremy Avigad. LeanArchitect: Automating blueprint generation for humans and AI. In *Interactive Theorem Proving (ITP 2026)*, volume 382 of *Leibniz International Proceedings in Informatics (LIPIcs)*, 2026. Paper 25.